Studio Nx

백엔드 151 — generate_reset_token()

2026-10-14

백엔드 151 — `generate_reset_token()` (재설정 토큰 만들기)

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-28 비밀번호 찾기 (FR-118): 두 번째 함수
작성	Claude (AI), 2026-10-14, 사용자 검토
이전	150 `hash_reset_token()`
시험	`tests/test_reset_token_hash.py`의 5개 (이 파일 전체 17개)
다음	요청 서비스(토큰을 만들어 저장하고 메일 링크를 내보냄)

1. 하는 일

새 재설정 토큰을 하나 만들어 **(원문, 해시)** 한 쌍을 돌려줍니다.

값	쓰임
원문 (43자)	메일 링크에만 넣습니다. 어디에도 저장하지 않습니다
해시 (64자)	`password_reset_tokens.token_hash`에 저장합니다

2. 코드

```
RESET_TOKEN_BYTES = 32  # 256비트

def generate_reset_token() -> tuple[str, str]:
    token = secrets.token_urlsafe(RESET_TOKEN_BYTES)
    return token, hash_reset_token(token)
```

3. 왜 이렇게

선택	이유
`secrets.token_urlsafe(32)`	운영체제의 암호용 난수로 무작위 32바이트(256비트)를 만들고, URL에 그대로 넣어도 되는 글자(`A-Z a-z 0-9 - _`) 43자로 바꿉니다. 추측이 불가능한 크기입니다
`random`을 쓰지 않음	`random`은 앞의 값으로 다음 값을 예측할 수 있습니다. 보안 토큰에는 쓰면 안 됩니다
원문과 해시를 한 쌍으로 돌려줌	호출하는 쪽이 해시를 따로 계산하다가 실수하는 일을 막고, 원문을 저장하지 않고 메일에만 쓰는 흐름을 함수 모양으로 드러냅니다
바이트 수를 상수로 뺌	`RESET_TOKEN_BYTES`로 이름을 붙여 시험과 문서가 같은 값을 가리키게 합니다

4. 시험 (5개 추가, 파일 전체 17개 통과)

확인	결과
돌려준 해시가 원문의 `hash_reset_token`과 같음	✓

확인	결과
원문이 43자이고 URL에 안전한 글자만 있음	✓
200번 만들어 원문과 해시가 모두 다름	✓
해시가 64자이고 원문을 포함하지 않음	✓
`secrets`를 32바이트로 불렀음 (`random`이 아님)	✓

전체 pytest 701개 통과, `ruff` 통과.